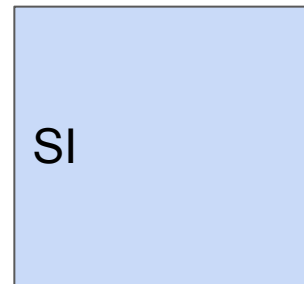


Scheduling on heterogeneous architectures

M. Mascheroni, A. Pérez-Calero for Submission Infrastructure

Three groups mainly involved



- PDMV prepares the request
- WMAgent translates the request into user jobs
- Submission Infrastructure provisions resources and executes the matchmaking

An SI oriented talk, but I'll do my best to include feedback from other groups as well

From GPU summary at the [PPD workshop](#)

Attributes from PdmV to WM

This needs to be precisely defined as it affects resource provisioning

Script to run before the pilot starts needs to be implemented/included, but no issues foreseen. Blackbox for SL.

HTCondor will advertise CUDAMaxSupportedVersion (as of 8.9.7, we use 8.8.12). This attribute is an integer representation of the highest CUDA version the machine's driver supports. HTCondor will no longer advertise the attribute CUDARuntimeVersion. ([Ticket #7413](#))

0. “Main switch” attribute:

a. GPUPreference:

- i. 0 - job does not need GPU
- ii. 1 - job would use GPU if is available
- iii. 2 - job requires GPU and it must be available

1. Mandatory attributes if GPU is preferred or needed:

- a. GPUMemory - (e.g. 8000, 16000) in MB
- b. CUDACapability - (e.g. 6.0, 7.5)
- c. CUDACompatibleRuntimes - (e.g. 11.1, 11.2, 11.3) - only a single value despite “Runtimes”

2. Optional/Debug attributes if GPU is preferred or needed:

- a. CUDARuntimeVersion (e.g. "11.1.74", "11.2.146") - the "native" or default version of the CUDA runtime running on the machine, if any
- b. CUDADriverVersion - (e.g. "460.32.03")
- c. GPUName - (e.g. "Tesla T4", "Quadro RTX 6000" ...)

Assuming .74 and .146 are typos

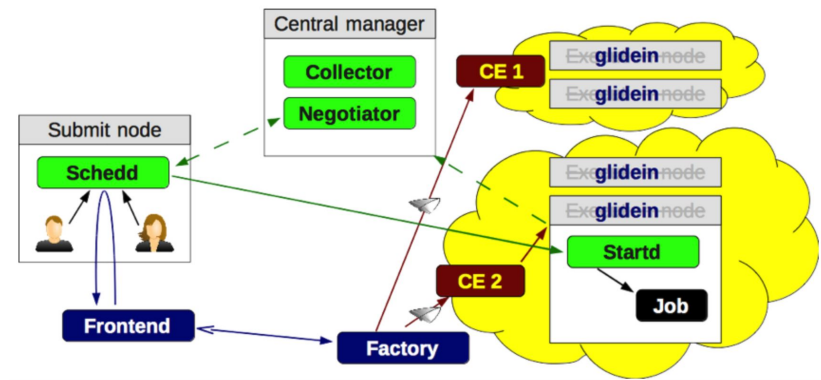
24/03/2021

Jordan Martins and Justinas Rumsevicius / GPU workflow steering

Most of these attributes already available in the global pool central manager. Just need to find the proper “mapping” (See backup slides 16 and 17)

Dynamic provisioning of resources and late binding

- **User jobs** are matched a first time to determine resource requests (aka **pilot jobs** or **glideins**):
 - A **static file** with the “description of the sites” is used for this purpose (maintained by factory operations)
 - Based on (1) user demand (jobs in the schedulers) and (2) pilot already running, **new pilot jobs are requested** (if necessary)
- **User jobs** are not linked to a specific pilot, they **can** actually **run everywhere**, and a pilot can run many user jobs. Therefore..
- **Second matchmaking** to actually start user jobs once pilot jobs successfully start (aka **late binding**)
 - Based on compatibility of resources to requests
 - Priority between user job are also considered here



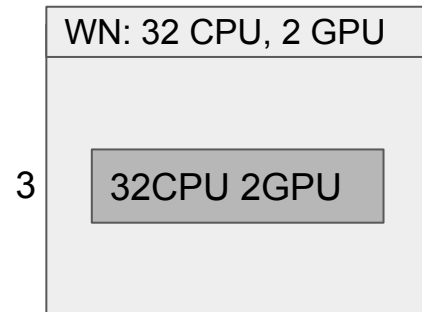
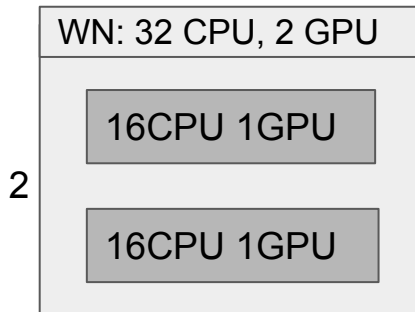
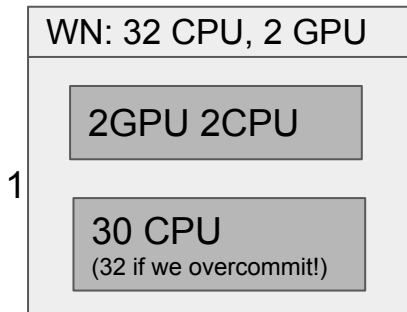
GPU scheduling: finding the proper GPU and CPU tweak

Resource provider perspective: GPU is the most expensive resource. Make sure GPU cycles are not wasted!

- Even if this means killing CPU jobs, or wasting CPU cores (leaving them idle)

Finding the proper **Node Configuration** is important:

1. Partition the WNs so that CPUs are treated separately?
2. Use big slots, and let condor do the matching?
3. Whole-node scheduling?



“Use GPU if available”: considerations I

Need to understand what this exactly means and why it is done.

- Clear **policies** about what **requestors** mean are needed
 - Are these jobs considered low priority “backfill”?
 - Will these jobs be the majority of the GPU requests or the minority?
 - Will they requests have a particular GPU/CPU/Memory pattern
- **Resource providers** have their own **policies** of use
- The two needs to be **combined**

“Use GPU if available”: considerations II

Policy decisions have implications on the implementations details. For example:

- Should a GPU pilot be requested (first matchmaking) if there is a GPU=maybe job?
 - The main condor knob to “trigger” a GPU pilot is *request_gpu=N*. Either the site has GPUs and we request a pilot, or it does not.
 - A fine balance needs to be found between not requesting enough GPU pilots, and wasting resources because we request too many pilots and workload is consumed too fast
- What to do when a GPU pilot starts (second matchmaking)
 - Should it execute CPU-only jobs on the excess of CPU resources?
 - Say a 2GPU/1CPU job starts on the whole node slots of the example before
 - The risk is we starve the GPU...
 - Do we give *GPU=yes* more priority than *GPU=maybe*?

GPU scheduling: final considerations

Need to find the best way of scheduling CPU only jobs on GPU pilots. Site policies matters a lot here (e.g.: they are the ones deciding if a whole node scheduling has to be used)

- Maybe considering **preemption**? Would be the **first time** we use it in SI

Also, need to minimize pilots landing on a GPU node and all the GPU jobs are gone (the pilot may sit in the site queue for days)

- But we already **remove pilots that are idle** in the site queues **if necessary**, so it should be fine

Conclusions

- GPU resources are being added to the global pool.
 - Opportunistic in nature: none of it is pledged so far
 - Already available for CRAB and CMSConnect users
- Submission infrastructure mostly focused on efficiency aspects
- Accounting of GPU usage to be clarified
 - In need of a GPU performance benchmark to measure used and allocated resources (?)

GPU as a topic for conferences

- Paper draft written for CHEP2021
 - We were advised to **complement the report with feedback from other CMS involved parties**
- Get in touch if you are interested!

Material

- GPU workflow steering talk at the PPD Workshop Spring 2021
- Github discussions:
 - <https://github.com/cms-sw/cms-sw/pull/33057>
 - <https://github.com/dmwm/WMCORE/issues/10393>
- Submission infrastructure dashboard: live GPU catalog
 - Periodically send GPU pilots to scan for resources

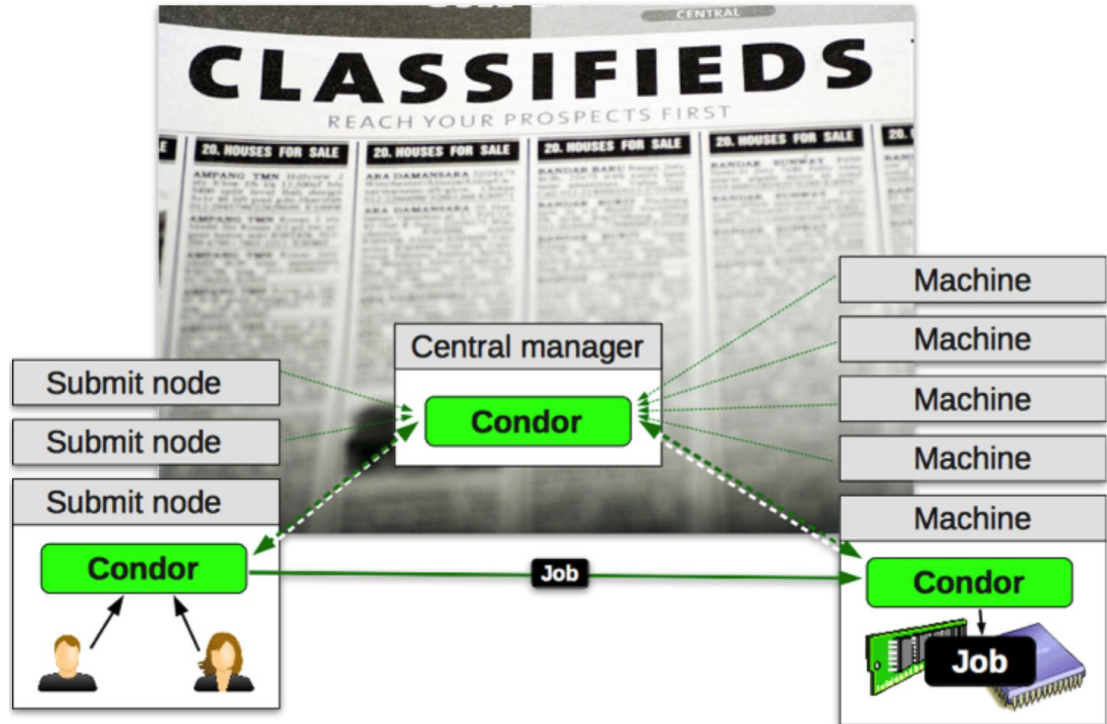
GPU resources

Site	Entry	CUDACapability	CUDAClockMhz	CUDAComputeUnits	CUDACoresPerCU	CUDADeviceName	CUDADriverVersion	CUDAECCEEnabled	CUDAGlobalMemoryMb
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit2_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit3_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.0	1329	56	64	Tesla P100-PCIE-16GB	11	true	16281
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.0	1329	56	64	Tesla P100-PCIE-12GB	11	true	12198
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.1	1582	28	128	GeForce GTX 1080 Ti	11	false	11178
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.1	1709	10	128	GeForce GTX 1060 6...	11	false	6078
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce6_gpu	6.1	1582	30	128	TITAN Xp	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce5_gpu	6.1	1582	30	128	TITAN Xp	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce6_gpu	6.1	1531	28	128	TITAN X (Pascal)	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce5_gpu	6.1	1531	28	128	TITAN X (Pascal)	10	false	12196
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.0	1380	80	64	Tesla V100-PCIE-32GB	11	true	32510
T2_US_Purdue	CMSHTPC_T2_US_Purdue_Hamme...	7.5	1590	40	64	Tesla T4	11	true	15110
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T3_US_OSG	Glow_US_Syracuse2_condor_gpu	7.5	1620	72	64	Quadro RTX 6000	11	true	22699
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.5	1770	72	64	Quadro RTX 8000	11	false	48601
T3_US_OSG	Glow_US_Syracuse3_condor_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T3_US_OSG	Glow_US_Syracuse2_condor_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110

Backup slides

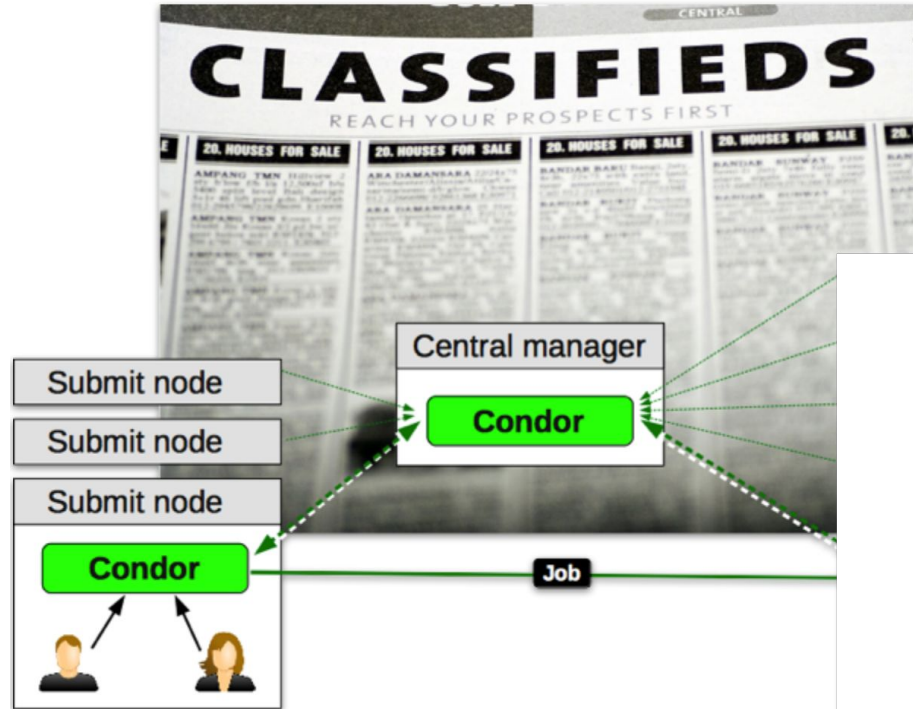
Global pool details

Condor pool works by matching **machine** classads (classified ads) and **jobs** classads



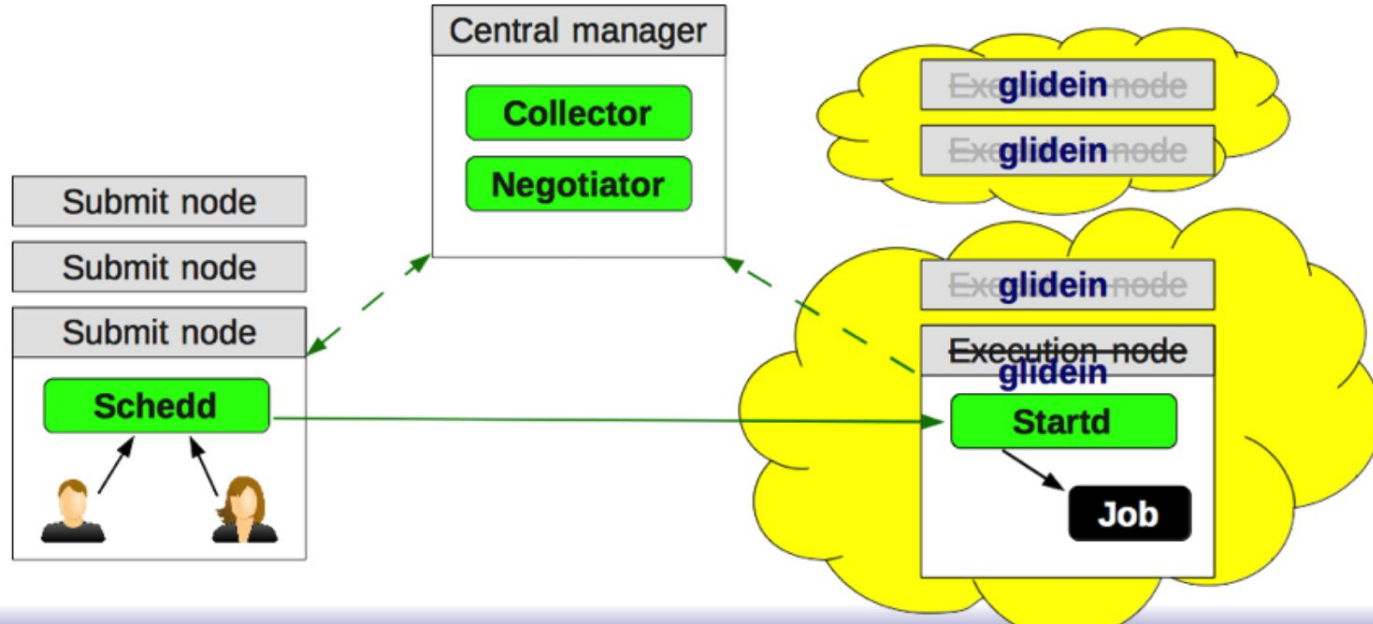
Global pool details

Machines are not always connected, they need to be **requested**



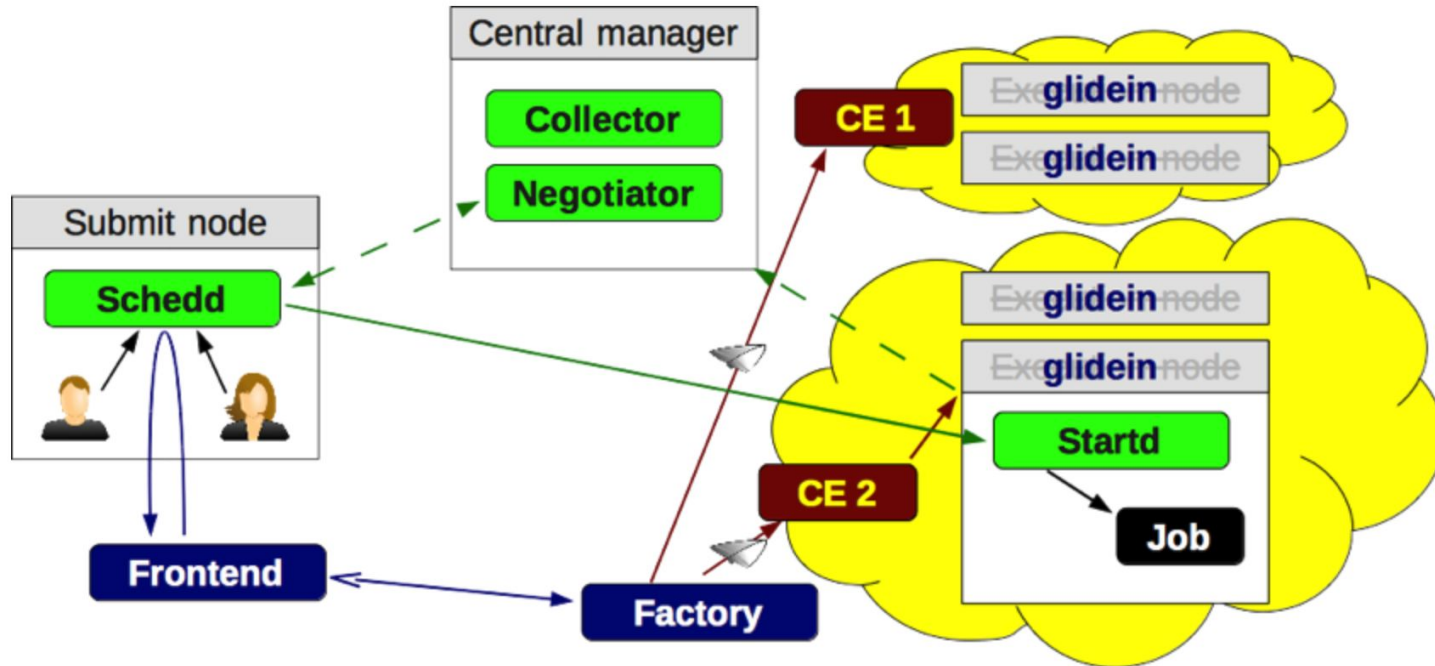
Global pool details

A pilot job (aka glidein) is a properly configured execution node submitted as a **grid job**



Global pool details

Two level matchmaking!



Querying the global pool for GPUs

```
[mmascher@lxplus775 ~]$ condor_status -pool vocms0814.cern.ch -const 'SlotType=="Partitionable" && GPUs!=undefined'
```

Name ActvtyTime	OpSys	Arch	State	Activity	LoadAv	Mem
slot1@glidein_5286_90490286@CRUSH-OSG-C7-GPU-10-5-6-39 0+06:32:48	LINUX	X86_64	Unclaimed	Idle	0.000	3072
slot1@glidein_16715_415043010@CRUSH-OSG-C7-GPU-10-5-229-69 slot1@glidein_13_130342560@f03-001-155-e.gridka.de 0+00:00:03	LINUX LINUX	X86_64 X86_64	Unclaimed Unclaimed	Idle Idle	0.000 0.000	3072 20000
slot1@glidein_14_71802184@f03-001-163-e.gridka.de 0+00:00:03	LINUX	X86_64	Unclaimed	Idle	0.000	20000
slot1@glidein_244055_214060700@hammer-f003.rcac.purdue.edu slot1@glidein_136865_886764516@hammer-f004.rcac.purdue.edu slot1@glidein_95306_897827582@hammer-f006.rcac.purdue.edu slot1@glidein_15720I_89522464@hammer-f007.rcac.purdue.edu slot1@glidein_61405_777826398@hammer-f011.rcac.purdue.edu slot1@glidein_16643I_561926952@hammer-f012.rcac.purdue.edu slot1@glidein_192484_209782656@hammer-f015.rcac.purdue.edu	LINUX LINUX LINUX LINUX LINUX LINUX LINUX	X86_64 X86_64 X86_64 X86_64 X86_64 X86_64 X86_64	Unclaimed Unclaimed Unclaimed Unclaimed Unclaimed Unclaimed Unclaimed	Idle Idle Idle Idle Idle Idle Idle	0.000 0.000 0.000 0.000 0.000 0.000 0.000	380466 141362 19634 274738 505394 80178 471602
						0+00:04:07 1+04:01:36 0+01:42:11 0+00:06:34 0+00:09:59 0+00:09:51 0+09:46:52
Machines	Owner	Claimed	Unclaimed	Matched	Preempting	Drain
X86_64/LINUX	11	0	0	11	0	0
Total	11	0	0	11	0	0

Don't put this in a script, CM is a critical component in SI and we try to keep queries to the minimum necessary ones!

Querying the global pool for GPU attributes

```
[mmascher@lxplus775 ~]$ condor_status -pool vocms0814.cern.ch -l  
slot1@glidein_244055_214060700@hammer-f003.rcac.purdue.edu | grep CUDA
```

```
AssignedGPUs = "CUDA0"  
CUDACapability = 7.5  
CUDAClockMhz = 1590.0  
CUDAComputeUnits = 40  
CUDACoresPerCU = 64  
CUDADeviceName = "Tesla T4"  
CUDADevicePciBusId = "0000:E2:00.0"  
CUDADeviceUuid = "a079a0d3-0ff7-90e0-e2d0-232504e79fa2"  
CUDADriverVersion = 11.0  
CUAECCEnabled = true  
CUDAGlobalMemoryMb = 15110
```

GPU Resources in the global pool

GPU resources									
Site	Entry	CUDACapability	CUDAClockMhz	CUDAComputeUnits	CUDACoresPerCU	CUDADeviceName	CUDADriverVersion	CUDA ECCEnabled	CUDAGlobalMemoryMb
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit2_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T2_US_Caltech	CMSHTPC_T2_US_Caltech_cit3_gpu	5.2	1216	24	128	GeForce GTX TITAN X	11	false	12213
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.0	1329	56	64	Tesla P100-PCIe-16GB	11	true	16281
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.0	1329	56	64	Tesla P100-PCIe-12GB	11	true	12198
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.1	1582	28	128	GeForce GTX 1080 Ti	11	false	11178
T3_US_OSG	HCC_US_Omaha_crane_gpu	6.1	1709	10	128	GeForce GTX 1060 6...	11	false	6078
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce6_gpu	6.1	1582	30	128	TITAN Xp	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce5_gpu	6.1	1582	30	128	TITAN Xp	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce6_gpu	6.1	1531	28	128	TITAN X (Pascal)	10	false	12196
T2_US_Vanderbilt	CMS_T2_US_Vanderbilt_ce5_gpu	6.1	1531	28	128	TITAN X (Pascal)	10	false	12196
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.0	1380	80	64	Tesla V100-PCIe-32GB	11	true	32510
T2_US_Purdue	CMSHTPC_T2_US_Purdue_Hamme...	7.5	1590	40	64	Tesla T4	11	true	15110
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T3_US_OSG	Glow_US_Syracuse2_condor_gpu	7.5	1620	72	64	Quadro RTX 6000	11	true	22699
T3_US_OSG	HCC_US_Omaha_crane_gpu	7.5	1770	72	64	Quadro RTX 8000	11	false	48601
T3_US_OSG	Glow_US_Syracuse3_condor_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T3_US_OSG	Glow_US_Syracuse2_condor_gpu	7.5	1815	48	64	Quadro RTX 5000	11	false	16125
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110
T2_US_Wisconsin	CMSHTPC_T2_US_Wisconsin_cms...	7.5	1590	40	64	Tesla T4	11	true	15110